

EAST WEST UNIVERSITY

Department of Computer Science and Engineering

CSE 487 - Cyber Security, Law, and Ethics

PROJECT REPORT

Design and Implementation of a Mini Security Operations Center (SOC) for Real-Time Threat Detection and Incident Response in an Isolated Virtual Environment

Submitted By	Shawna Akter ID: 2023-1-60-071
Submitted To	Rakib Hossen Assistant Professor, Dept. of CSE East West University
Course	CSE 487 - Cyber Security, Law, and Ethics
Submission Date	22 August, 2026

Ethical Hacking Final Assignment

Contents

1. Abstract
2. Introduction
3. Literature Review
4. Methodology
5. Lab Environment and Architecture
6. Implementation
7. Results and Discussion
8. Challenges and Resolutions
9. Security Recommendations
10. Conclusion
11. References

Project Scope and Safety Statement

All scanning, authentication testing, monitoring, and firewall validation described in this report were carried out only inside an authorized VirtualBox laboratory. No public, institutional, or third-party system was targeted.

1. Abstract

This project presents the design and implementation of a compact Security Operations Center (SOC) laboratory for centralized security monitoring, threat detection, event correlation, and incident response. The environment was built in VirtualBox using an Ubuntu 24.04.4 LTS virtual machine as the central Wazuh server, a Windows 11 endpoint monitored through the Wazuh agent and Microsoft Sysmon, and a Kali Linux virtual machine used only for controlled security testing. Wazuh 4.14.7 provided the manager, indexer, and dashboard functions, while Suricata 8.0.6 supplied network intrusion-detection telemetry. File Integrity Monitoring (FIM), Sysmon event collection, network reconnaissance detection, SSH authentication-failure detection, and a Windows failed-logon event were evaluated. A defensive response was then demonstrated by blocking the Kali source address with UFW and re-testing SSH access. The resulting timeout confirmed that the applied control prevented the previously permitted connection. The completed laboratory therefore demonstrates a practical SOC workflow of monitoring, detection, investigation, mitigation, and validation within a safe and isolated environment.

2. Introduction

Security Operations Centers are used to collect and analyze security telemetry from endpoints, servers, and network controls so that suspicious behavior can be identified and investigated in a timely manner. Instead of depending on a single defensive tool, a SOC combines log collection, detection rules, dashboards, endpoint telemetry, network monitoring, and incident response into one operational workflow.

The objective of this project was to build a small but functional SOC that demonstrates those concepts in a form appropriate for an ethical-hacking laboratory. The project was intentionally implemented in an isolated virtual environment to satisfy the course requirement that all practical testing remain authorized and controlled. The final design combined Wazuh, Sysmon, Suricata, Kali Linux, Windows event logs, and UFW firewall controls. The implementation also emphasized evidence-based validation: every major phase was verified using service status, dashboard output, alert counts, event records, or before-and-after connectivity tests.

2.1 Project Objectives

- Deploy a centralized Wazuh SOC platform for event collection, indexing, and visualization.
- Enroll a Windows endpoint and collect security telemetry from it.
- Integrate Sysmon for detailed Windows activity monitoring.
- Demonstrate File Integrity Monitoring for controlled file modification.
- Integrate Suricata IDS events with Wazuh.
- Generate controlled reconnaissance and authentication-failure activities from Kali Linux.
- Validate a basic incident-response control using UFW and re-test the blocked service.

3. Literature Review

The implementation was guided primarily by vendor and framework documentation because the project focuses on reproducible security engineering and tool integration rather than development of a new detection algorithm.

Wazuh documentation describes an architecture consisting of endpoint agents and three central components: the Wazuh server, indexer, and dashboard. Endpoint data are collected by agents, analyzed by the server, indexed for storage and search, and visualized in the dashboard. The documentation also identifies the all-in-one deployment as suitable for laboratories and small environments, which directly matches the scale of this project [1].

Microsoft Sysmon is a persistent Windows service and driver that records detailed system activity, including process creation and other security-relevant events, in the Microsoft-Windows-Sysmon/Operational event

channel. These events become useful to a SIEM when they are centrally collected and analyzed [2]. This motivated the integration of Sysmon with the Windows Wazuh agent.

Suricata provides an extensible EVE JSON output that can include alerts, anomalies, metadata, file information, and protocol-specific records in a common JSON event stream [3]. The project therefore configured Wazuh to ingest Suricata's eve.json output, allowing network IDS events to be presented in the same SOC interface as endpoint events.

The MITRE ATT&CK framework classifies repeated credential guessing under Brute Force (T1110), including Password Guessing (T1110.001). MITRE notes that SSH is one of the commonly targeted management services and identifies repeated failed SSH authentications as a relevant detection pattern [4]. This framing aligns with the authentication-failure experiment used in the project.

4. Methodology

The project used a staged implementation methodology so that each security layer could be validated before the next one was introduced. The overall workflow was:

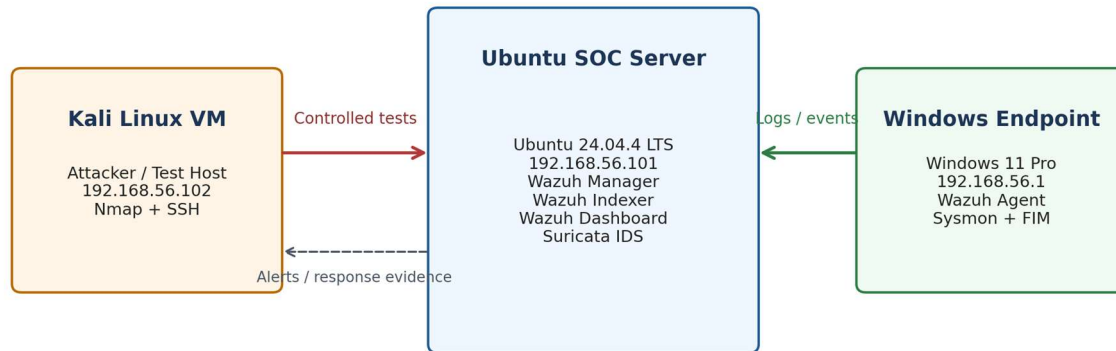
1. Prepare an isolated VirtualBox network and allocate sufficient CPU, RAM, and disk resources.
2. Install and validate Wazuh central components on Ubuntu.
3. Connect the Windows endpoint through a Wazuh agent.
4. Add Sysmon and File Integrity Monitoring telemetry.
5. Install Suricata and forward its EVE JSON events into Wazuh.
6. Prepare Kali Linux as an authorized test host on the same host-only subnet.
7. Generate controlled reconnaissance and failed-authentication events.
8. Investigate the resulting SOC alerts and MITRE ATT&CK mappings.
9. Apply a UFW deny rule as a response action and repeat the SSH connection test.

5. Lab Environment and Architecture

Component	Role	Operating System / Version	Host-Only IP	Key Tools
SOC Server	Central SIEM / IDS collector	Ubuntu 24.04.4 LTS	192.168.56.101	Wazuh 4.14.7, Suricata 8.0.6, UFW, OpenSSH
Endpoint	Monitored workstation	Windows 11 Pro	192.168.56.1	Wazuh Agent 4.14.7, Sysmon, Windows Event Log
Test Host	Authorized attacker / test VM	Kali Linux 2026.2	192.168.56.102	Nmap, OpenSSH client

The Ubuntu VM was configured with 8 GB RAM, 4 virtual CPUs, and a 50 GB virtual disk. Two network adapters were used: NAT for package downloads and a Host-Only adapter for isolated SOC communication. The Kali VM used the same Host-Only network for controlled tests. The Windows host communicated with the Ubuntu VM through the VirtualBox Host-Only Ethernet Adapter.

Isolated VirtualBox SOC Laboratory Architecture



Host-Only network: 192.168.56.0/24 | NAT used only for software downloads/updates

Figure A: Logical architecture of the isolated Mini SOC laboratory.

6. Implementation

6.1 Wazuh Central Platform Deployment

Wazuh was installed as an all-in-one deployment on Ubuntu. The final successful installation initialized the Wazuh manager, indexer, and dashboard. The dashboard was exposed over HTTPS on TCP/443, while agent communication and enrollment used TCP/1514 and TCP/1515.

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

```
21/08/2026 17:15:21 INFO: Starting Wazuh dashboard installation.
21/08/2026 17:23:42 INFO: Wazuh dashboard installation finished.
21/08/2026 17:23:42 INFO: Wazuh dashboard post-install configuration finished.
21/08/2026 17:23:42 INFO: Starting service wazuh-dashboard.
21/08/2026 17:23:43 INFO: wazuh-dashboard service started.
21/08/2026 17:23:45 INFO: Updating the internal users.
21/08/2026 17:23:53 INFO: A backup of the internal users has been saved in the /
etc/wazuh-indexer/internalusers-backup folder.
21/08/2026 17:24:31 INFO: The filebeat.yml file has been updated to use the File
beat Keystore username and password.
21/08/2026 17:25:19 INFO: Initializing Wazuh dashboard web application.
21/08/2026 17:25:19 INFO: Wazuh dashboard web application not yet initialized. W
aiting...
21/08/2026 17:25:35 INFO: Wazuh dashboard web application not yet initialized. W
aiting...
21/08/2026 17:25:50 INFO: Wazuh dashboard web application initialized.
21/08/2026 17:25:50 INFO: --- Summary ---
21/08/2026 17:25:50 INFO: You can access the web interface https://<wazuh-dashbo
ard-ip>:443
  User: admin
  Password Password redacted for reportsecurity
21/08/2026 17:25:50 INFO: Installation finished.
  ^C
```

Figure 1: Successful Wazuh 4.14.7 installation and dashboard initialization (credential redacted).

The central services and expected listening ports were then verified:

```
sudo systemctl is-active wazuh-manager
sudo systemctl is-active wazuh-indexer
sudo systemctl is-active wazuh-dashboard
sudo ss -ltnp | egrep ':443|:1514|:1515'
ip -br a
```

```
sudo systemctl is-active wazuh-manager
sudo systemctl is-active wazuh-indexer
sudo systemctl is-active wazuh-dashboard
[sudo] password for :
active
active
active

sudo ss -ltnp | egrep ':443|:1514|:1515'
LISTEN 0      128          0.0.0.0:*    users:((("wazuh-r
emoted",pid=64368,fd=4))
LISTEN 0      128          0.0.0.0:*    users:((("wazuh-a
uthd",pid=64222,fd=6))
LISTEN 0      511          0.0.0.0:*    users:((("node",p
id=65702,fd=19))

ip -br a
lo                UNKNOWN    127.0.0.1/8 ::1/128
enp0s3            UP          10.0.2.15/24 fd17:625c:f037:2:c24a:4ef3:8c97:5b3
7/64 fd17:625c:f037:2:a00:27ff:fefc:ef24/64 fe80::a00:27ff:fefc:ef24/64
enp0s8            UP          192.168.56.101/24 fe80::d44d:1282:b8b9:7d45/64
```

Figure 2: Wazuh services active, required ports listening, and SOC server IP 192.168.56.101.

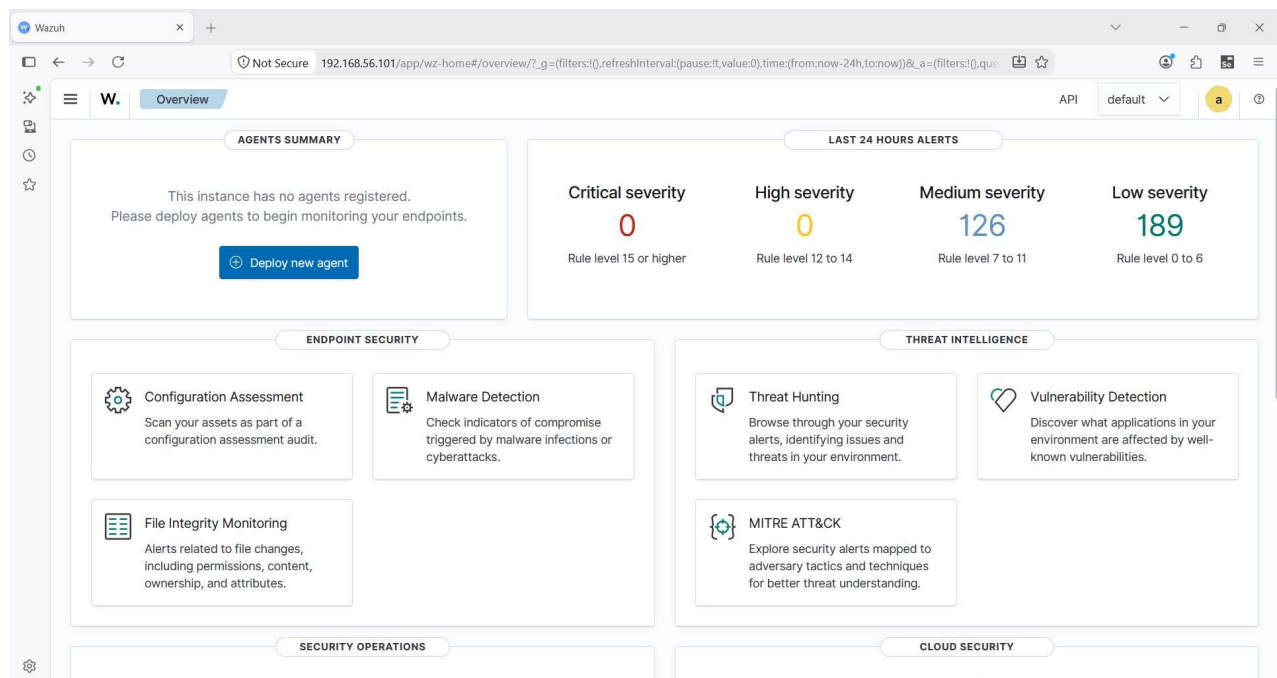


Figure 3: Initial Wazuh dashboard confirming successful web access to the SOC platform.

6.2 Windows Endpoint Enrollment

Before installing the endpoint agent, connectivity from Windows to the Ubuntu Wazuh server was verified on the required ports.

```
Test-NetConnection 192.168.56.101 -Port 1514
Test-NetConnection 192.168.56.101 -Port 1515
Test-NetConnection 192.168.56.101 -Port 443
```

```
Administrator: Windows PowerShell

PS C:\WINDOWS\system32> Test-NetConnection 192.168.56.101 -Port 1514
>> Test-NetConnection 192.168.56.101 -Port 1515
>> Test-NetConnection 192.168.56.101 -Port 443

ComputerName      : 192.168.56.101
RemoteAddress     : 192.168.56.101
RemotePort        : 1514
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.56.1
TcpTestSucceeded  : True

ComputerName      : 192.168.56.101
RemoteAddress     : 192.168.56.101
RemotePort        : 1515
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.56.1
TcpTestSucceeded  : True

ComputerName      : 192.168.56.101
RemoteAddress     : 192.168.56.101
RemotePort        : 443
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.56.1
TcpTestSucceeded  : True

PS C:\WINDOWS\system32>
```

Figure 4: Successful Windows connectivity tests to the Wazuh agent, enrollment, and dashboard ports.

The Wazuh Windows agent was then installed and pointed to the manager at 192.168.56.101.

```
New-Item -ItemType Directory -Path C:\WazuhSetup -Force
Set-Location C:\WazuhSetup
Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi" -OutFile
"wazuh-agent-4.14.7-1.msi"
Start-Process msixexec.exe -Wait -ArgumentList '/i "C:\WazuhSetup\wazuh-agent-4.14.7-1.msi" /q
WAZUH_MANAGER="192.168.56.101" WAZUH_AGENT_NAME="Windows-Endpoint"'
Start-Service WazuhSvc
```

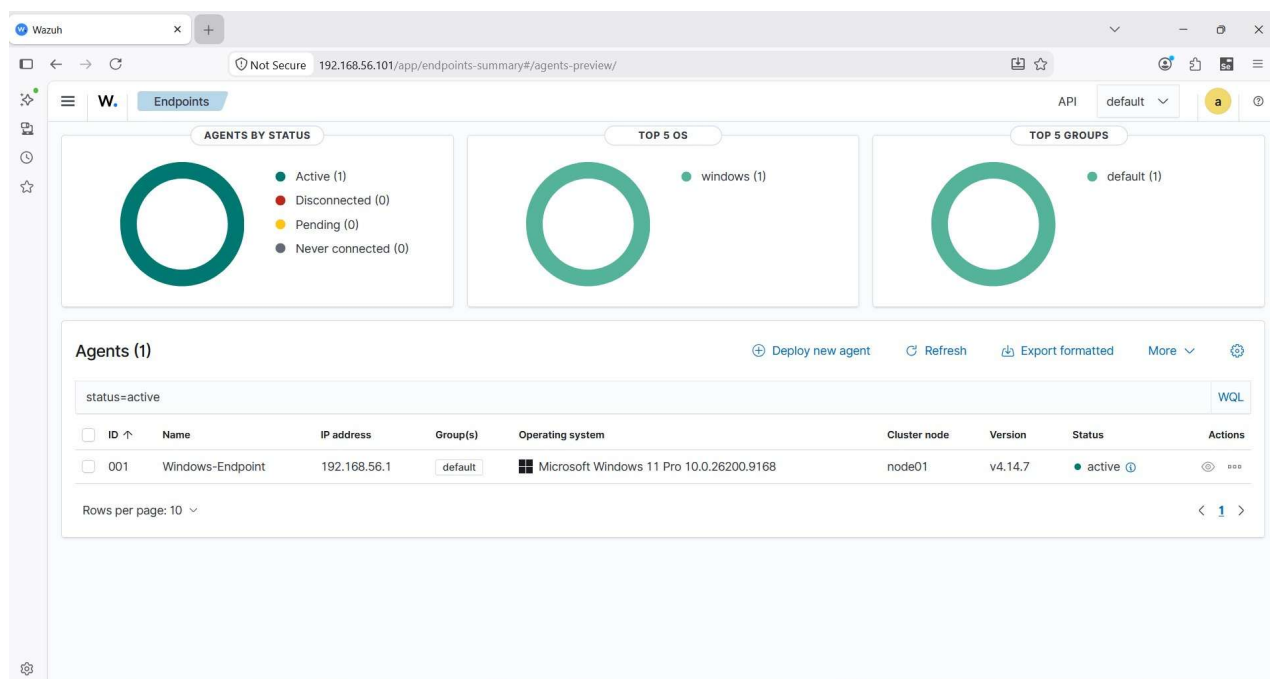
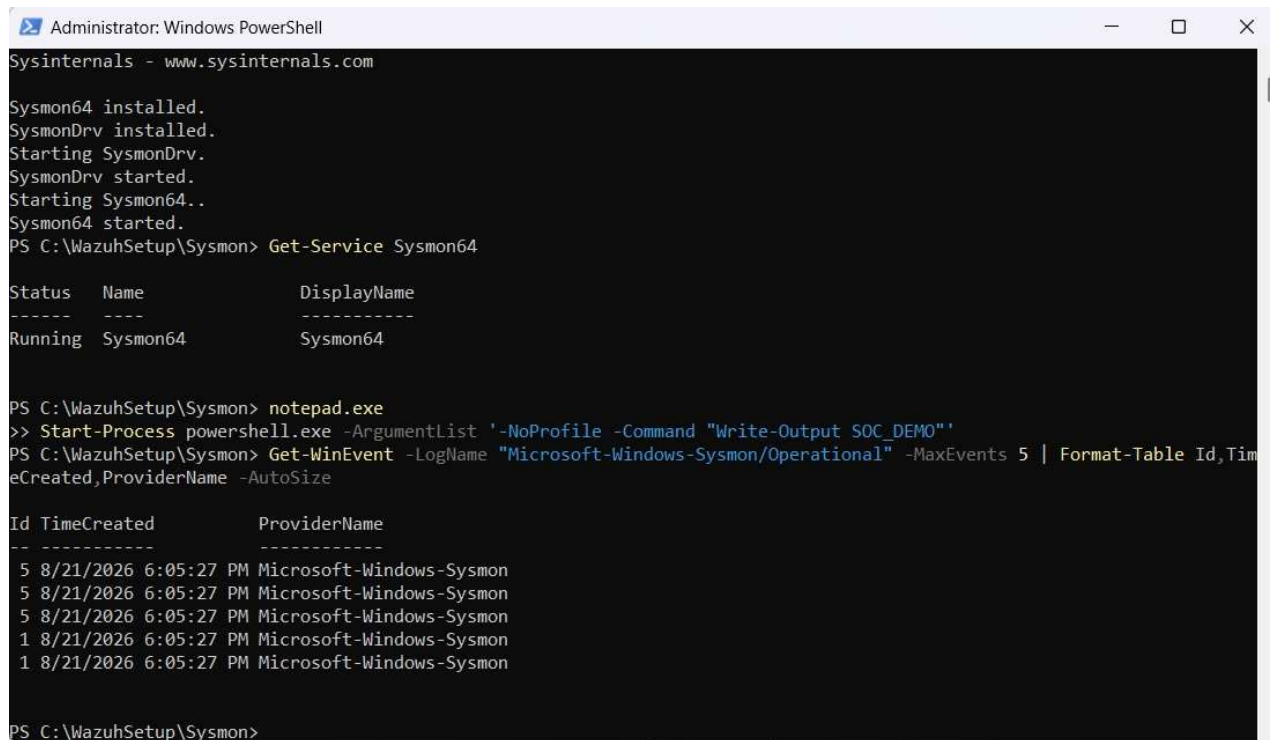


Figure 5: Windows-Endpoint successfully enrolled and shown as Active in Wazuh.

6.3 Sysmon Integration

Sysmon was installed on the Windows endpoint to produce richer host telemetry. A harmless Notepad and PowerShell activity was generated to verify that Sysmon events were being created.

```
Invoke-WebRequest -Uri "https://download.sysinternals.com/files/Sysmon.zip" -OutFile "Sysmon.zip"
Expand-Archive .\Sysmon.zip -DestinationPath .\Sysmon -Force
Set-Location .\Sysmon
.\Sysmon64.exe -accepteula -i
Get-Service Sysmon64
notepad.exe
Start-Process powershell.exe -ArgumentList '-NoProfile -Command "Write-Output SOC_DEMO"'
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5 | Format-Table
Id,TimeCreated,ProviderName -AutoSize
```



The screenshot shows a Windows PowerShell window titled "Administrator: Windows PowerShell". The prompt is "Sysinternals - www.sysinternals.com". The user has entered several commands to install and start Sysmon, open Notepad, and generate events. The output shows Sysmon64 installed, SysmonDrv installed, and Sysmon64 started. A table of Sysmon events is displayed, showing five events with Id, TimeCreated, and ProviderName.

```
PS C:\WazuhSetup\Sysmon> Get-Service Sysmon64
```

Status	Name	DisplayName
Running	Sysmon64	Sysmon64

```
PS C:\WazuhSetup\Sysmon> notepad.exe
>> Start-Process powershell.exe -ArgumentList '-NoProfile -Command "Write-Output SOC_DEMO"'
PS C:\WazuhSetup\Sysmon> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5 | Format-Table Id,TimeCreated,ProviderName -AutoSize
```

Id	TimeCreated	ProviderName
5	8/21/2026 6:05:27 PM	Microsoft-Windows-Sysmon
5	8/21/2026 6:05:27 PM	Microsoft-Windows-Sysmon
5	8/21/2026 6:05:27 PM	Microsoft-Windows-Sysmon
1	8/21/2026 6:05:27 PM	Microsoft-Windows-Sysmon
1	8/21/2026 6:05:27 PM	Microsoft-Windows-Sysmon

```
PS C:\WazuhSetup\Sysmon>
```

Figure 6: Sysmon service running and recent Sysmon events generated on the Windows endpoint.

The Sysmon event channel was added to the Wazuh agent configuration and the agent service was restarted so that events could be forwarded to the SOC.

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>

Restart-Service WazuhSvc
```

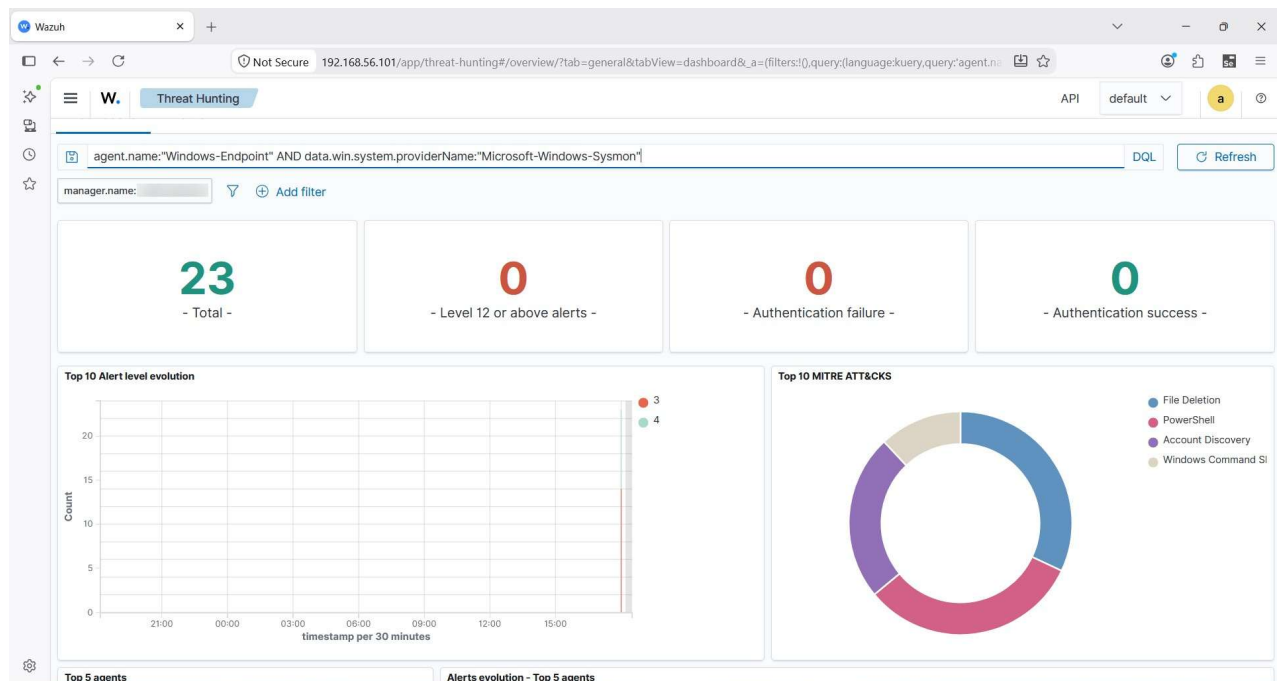


Figure 7: Wazuh Threat Hunting view showing Sysmon events and MITRE ATT&CK categories from Windows-Endpoint.

6.4 File Integrity Monitoring

A dedicated test directory was used for a controlled FIM demonstration. Wazuh was configured to monitor the directory in real time and report content changes.

```
New-Item -ItemType Directory -Path C:\SOC-Demo -Force
"Original SOC baseline content" | Set-Content C:\SOC-Demo\important.txt

<syscheck>
  <directories realtime="yes" check_all="yes" report_changes="yes">C:\SOC-Demo</directories>
</syscheck>

Restart-Service WazuhSvc
Add-Content C:\SOC-Demo\important.txt "Controlled SOC modification"
```

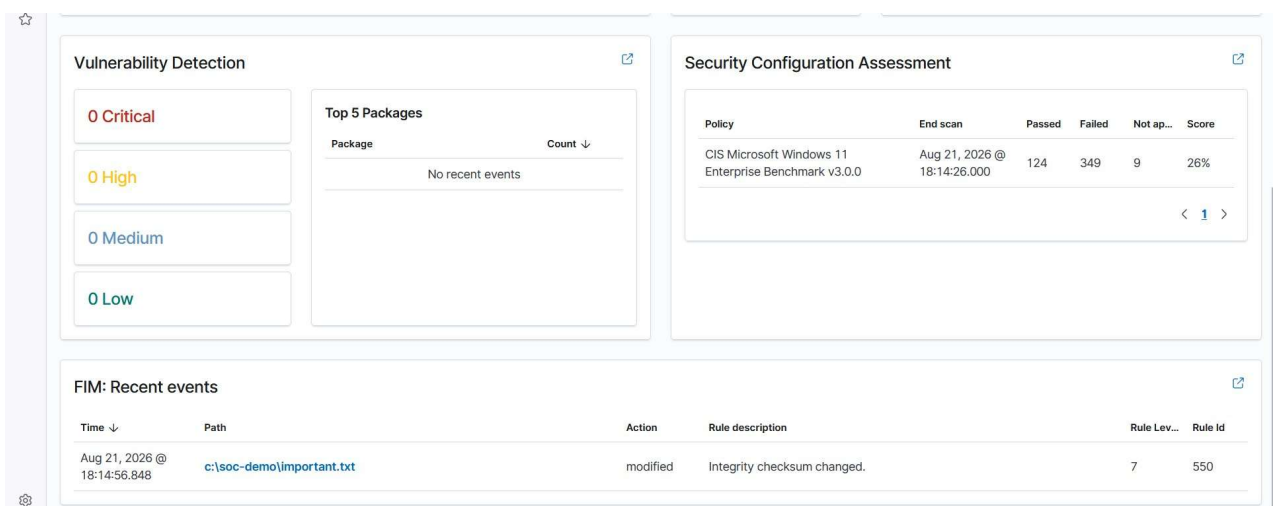


Figure 8: FIM event for c:\soc-demo\important.txt showing a modification and integrity checksum change (Rule ID 550, level 7).

6.5 Suricata IDS Deployment and Wazuh Integration

Suricata was installed on the Ubuntu SOC server and run on the Host-Only interface enp0s8. A configuration test was completed before starting the IDS in live mode.

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:oisf/suricata-stable
sudo apt update
sudo apt install -y suricata
sudo suricata-update
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl stop suricata
sudo suricata -D -i enp0s8 -c /etc/suricata/suricata.yaml
```

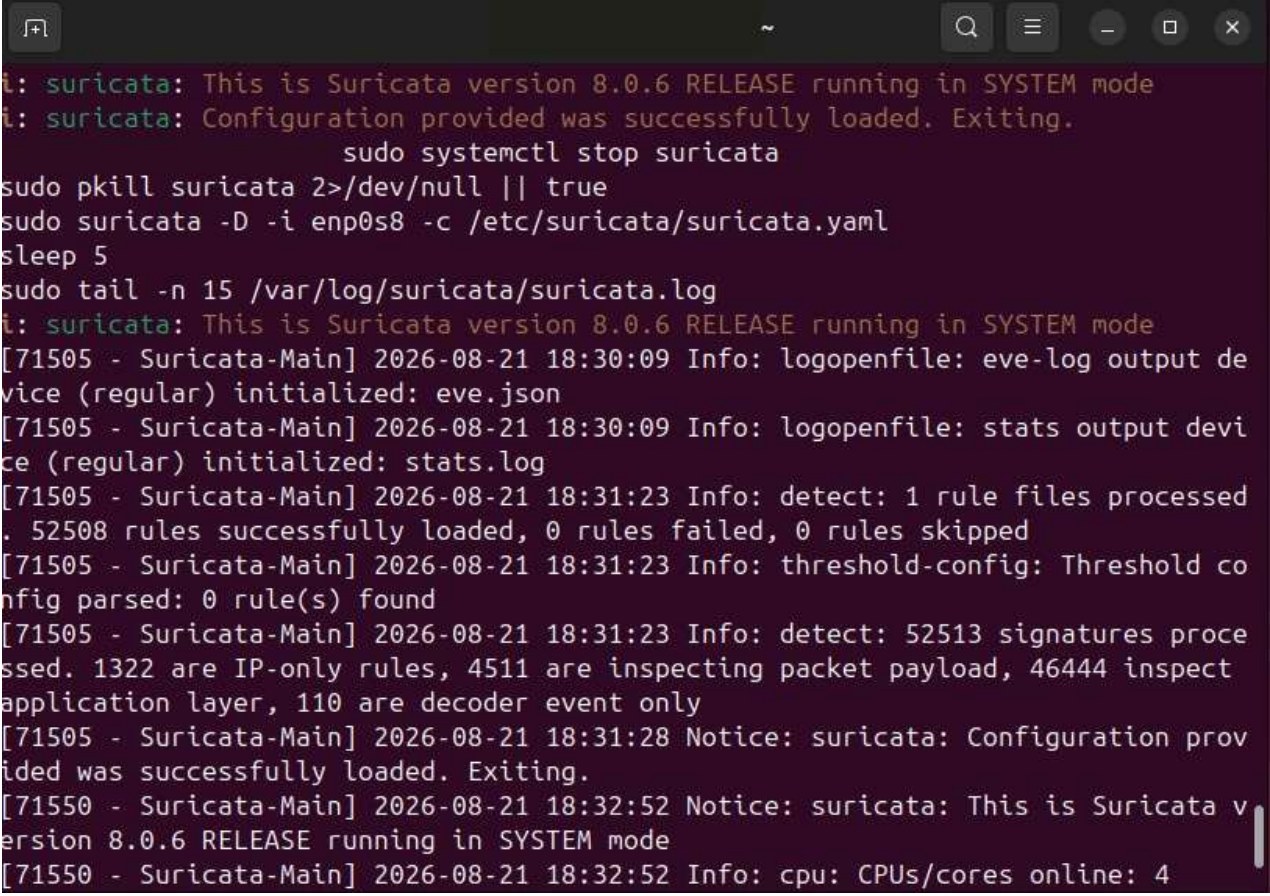
A terminal window with a dark background and light-colored text. The window title bar shows standard Linux window controls. The terminal output shows the following sequence of commands and messages:
1. Initial command: `sudo suricata -D -i enp0s8 -c /etc/suricata/suricata.yaml`
2. Message: `i: suricata: This is Suricata version 8.0.6 RELEASE running in SYSTEM mode`
3. Message: `i: suricata: Configuration provided was successfully loaded. Exiting.`
4. Command: `sudo systemctl stop suricata`
5. Command: `sudo pkill suricata 2>/dev/null || true`
6. Command: `sudo suricata -D -i enp0s8 -c /etc/suricata/suricata.yaml`
7. Command: `sleep 5`
8. Command: `sudo tail -n 15 /var/log/suricata/suricata.log`
9. Log output (from the tail command):
- `i: suricata: This is Suricata version 8.0.6 RELEASE running in SYSTEM mode`
- `[71505 - Suricata-Main] 2026-08-21 18:30:09 Info: logopenfile: eve-log output device (regular) initialized: eve.json`
- `[71505 - Suricata-Main] 2026-08-21 18:30:09 Info: logopenfile: stats output device (regular) initialized: stats.log`
- `[71505 - Suricata-Main] 2026-08-21 18:31:23 Info: detect: 1 rule files processed. 52508 rules successfully loaded, 0 rules failed, 0 rules skipped`
- `[71505 - Suricata-Main] 2026-08-21 18:31:23 Info: threshold-config: Threshold config parsed: 0 rule(s) found`
- `[71505 - Suricata-Main] 2026-08-21 18:31:23 Info: detect: 52513 signatures processed. 1322 are IP-only rules, 4511 are inspecting packet payload, 46444 inspect application layer, 110 are decoder event only`
- `[71505 - Suricata-Main] 2026-08-21 18:31:28 Notice: suricata: Configuration provided was successfully loaded. Exiting.`
- `[71550 - Suricata-Main] 2026-08-21 18:32:52 Notice: suricata: This is Suricata version 8.0.6 RELEASE running in SYSTEM mode`
- `[71550 - Suricata-Main] 2026-08-21 18:32:52 Info: cpu: CPUs/cores online: 4`

Figure 9: Suricata 8.0.6 running in IDS/live mode with EVE logging enabled and rules successfully loaded.

Wazuh was then configured to ingest Suricata's EVE JSON event stream.

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
```

```
signal handling
[71551 - Suricata-Main] 2026-08-21 18:32:52 Info: conf: Running in live mode, activating unix socket
[71551 - Suricata-Main] 2026-08-21 18:32:52 Info: logopenfile: fast output device (regular) initialized: fast.log
[71551 - Suricata-Main] 2026-08-21 18:32:52 Info: logopenfile: eve-log output device (regular) initialized: eve.json
[71551 - Suricata-Main] 2026-08-21 18:32:52 Info: logopenfile: stats output device (regular) initialized: stats.log
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.pre-suricata
sudo tee -a /var/ossec/etc/ossec.conf >/dev/null <<'EOF'

<ossec_config>
  <localfile>
    <log_format>json</log_format>
    <location>/var/log/suricata/eve.json</location>
  </localfile>
</ossec_config>
EOF

sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
active
```

Figure 10: Suricata EVE JSON integration added to Wazuh and Wazuh manager confirmed active.

6.6 Kali Linux Test Host and Network Validation

Kali Linux was configured with a Host-Only address of 192.168.56.102. Connectivity to the Ubuntu SOC server at 192.168.56.101 was verified before any security test was performed.

```
ip -br a
ping -c 3 192.168.56.101
```

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ip -br a  
lo                UNKNOWN    127.0.0.1/8 ::1/128  
eth0              UP          10.0.2.15/24 fd17:625c:f037:2:c783:81a9:5dd8:  
b71e/64 fe80::48b2:f170:d40f:dd87/64  
eth1              UP          192.168.56.102/24 fe80::86bd:510c:cfb9:2dfd/6  
4  
  
(kali@kali)-[~]  
$ ping -c 3 192.168.56.101  
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.  
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=3.34 ms  
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=1.14 ms  
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=2.42 ms  
  
— 192.168.56.101 ping statistics —  
3 packets transmitted, 3 received, 0% packet loss, time 2506ms  
rtt min/avg/max/mdev = 1.140/2.299/3.342/0.902 ms  
  
(kali@kali)-[~]  
$
```

Figure 11: Kali Host-Only interface 192.168.56.102 and successful reachability test to the SOC server.

6.7 Controlled Network Reconnaissance Test

A SYN scan was run only against the authorized Ubuntu VM. The scan showed TCP/443 open while most tested ports were filtered, providing a controlled reconnaissance event for the IDS/SIEM pipeline.

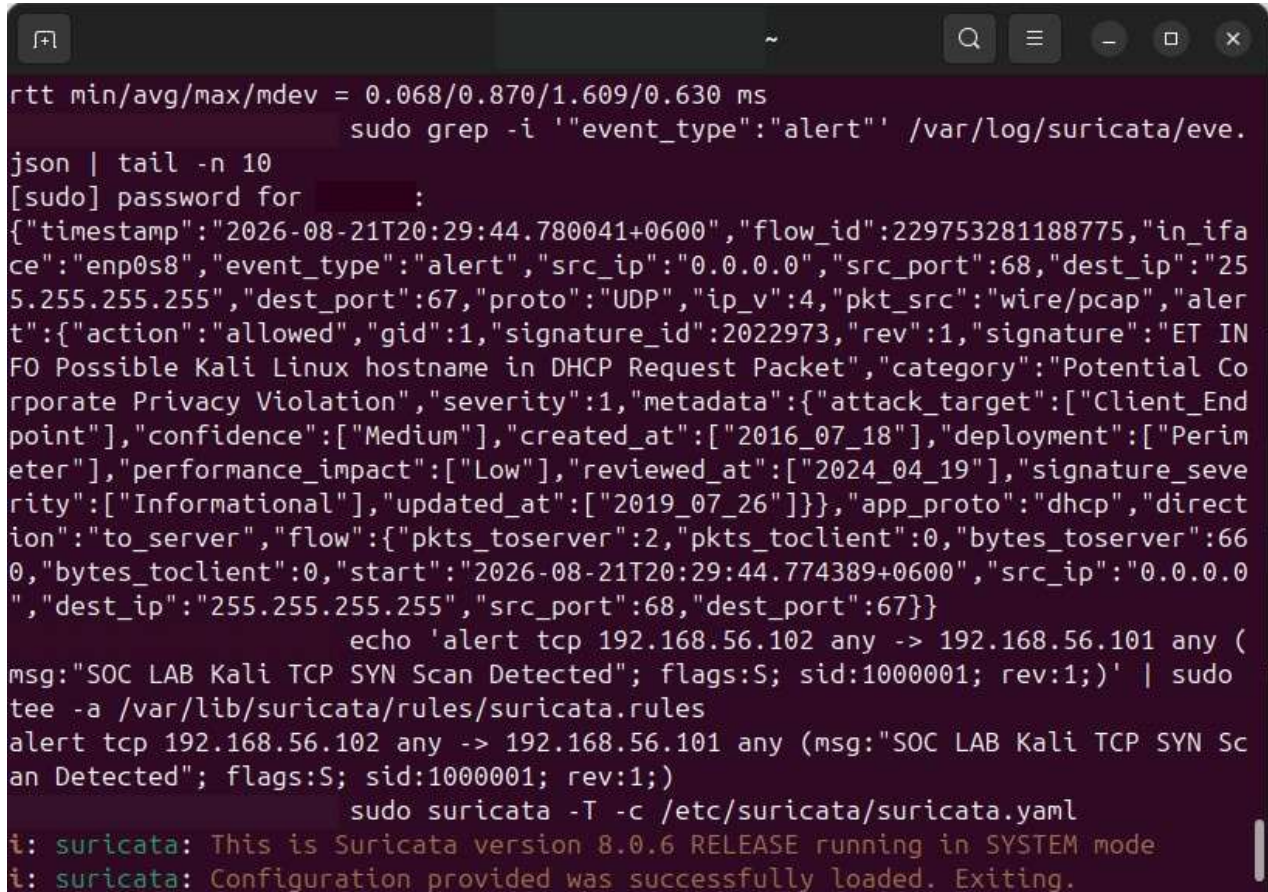
```
sudo nmap -sS -Pn -T4 --top-ports 1000 192.168.56.101
```

```
(kali@kali)-[~]  
$ sudo nmap -sS -Pn -T4 --top-ports 1000 192.168.56.101  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-21 10:43 -0400  
Nmap scan report for 192.168.56.101  
Host is up (0.0013s latency).  
Not shown: 996 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
22/tcp    closed ssh  
53/tcp    closed domain  
80/tcp    closed http  
443/tcp   open  https  
MAC Address: 08:00:27:F9:5C:95 (Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 5.28 seconds  
  
(kali@kali)-[~]  
$
```

Figure 12: Controlled Nmap SYN reconnaissance against 192.168.56.101.

Suricata initially generated an informational Kali-related alert. A dedicated local rule was also used in the lab to make the SYN-scan detection explicit and reproducible.

```
echo 'alert tcp 192.168.56.102 any -> 192.168.56.101 any (msg:"SOC LAB Kali TCP SYN Scan Detected"; flags:S; sid:1000001; rev:1;)' | sudo tee -a /var/lib/suricata/rules/suricata.rules
sudo suricata -T -c /etc/suricata/suricata.yaml
```

A terminal window with a dark background and light-colored text. The output shows network statistics (rtt min/avg/max/mdev), a command to grep for 'alert' events in the Suricata log, and a detailed JSON alert. The alert is for a DHCP request from 0.0.0.0 to 255.255.255.255 on port 67, categorized as a 'Potential Corporate Privacy Violation'. Below the alert, the custom rule configuration is shown, followed by the Suricata startup messages indicating it is running in SYSTEM mode with the provided configuration loaded successfully.

```
rtt min/avg/max/mdev = 0.068/0.870/1.609/0.630 ms
sudo grep -i '"event_type":"alert"' /var/log/suricata/eve.
json | tail -n 10
[sudo] password for :
{"timestamp":"2026-08-21T20:29:44.780041+0600","flow_id":229753281188775,"in_iface":"enp0s8","event_type":"alert","src_ip":"0.0.0.0","src_port":68,"dest_ip":"255.255.255.255","dest_port":67,"proto":"UDP","ip_v":4,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":2022973,"rev":1,"signature":"ET INFO Possible Kali Linux hostname in DHCP Request Packet","category":"Potential Corporate Privacy Violation","severity":1,"metadata":{"attack_target":["Client_Endpoint"],"confidence":["Medium"],"created_at":["2016_07_18"],"deployment":["Perimeter"],"performance_impact":["Low"],"reviewed_at":["2024_04_19"],"signature_severity":["Informational"],"updated_at":["2019_07_26"]},"app_proto":"dhcp","direction":"to_server","flow":{"pkts_toserver":2,"pkts_toclient":0,"bytes_toserver":660,"bytes_toclient":0,"start":"2026-08-21T20:29:44.774389+0600","src_ip":"0.0.0.0","dest_ip":"255.255.255.255","src_port":68,"dest_port":67}}
echo 'alert tcp 192.168.56.102 any -> 192.168.56.101 any (msg:"SOC LAB Kali TCP SYN Scan Detected"; flags:S; sid:1000001; rev:1;)' | sudo tee -a /var/lib/suricata/rules/suricata.rules
alert tcp 192.168.56.102 any -> 192.168.56.101 any (msg:"SOC LAB Kali TCP SYN Scan Detected"; flags:S; sid:1000001; rev:1;)
sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 8.0.6 RELEASE running in SYSTEM mode
i: suricata: Configuration provided was successfully loaded. Exiting.
```

Figure 13: Suricata alert evidence and custom lab rule used for explicit Kali SYN-scan detection.

6.8 SSH Authentication-Failure Detection

An SSH test account was created on Ubuntu. From Kali, several incorrect passwords were entered before one legitimate login was performed. Wazuh detected the failed authentications and mapped the activity to password-guessing / SSH behavior.

```
# Ubuntu
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp
sudo useradd -m -s /bin/bash socdemo
echo 'socdemo:<LAB_PASSWORD>' | sudo chpasswd

# Kali
ssh socdemo@192.168.56.101

sudo grep -Ei 'Failed password|Accepted password' /var/log
/ssh.log | tail -n 20
grep: /var/log/ssh.log: binary file matches
```

Figure 14: Ubuntu authentication-log verification performed after the controlled SSH attempts.

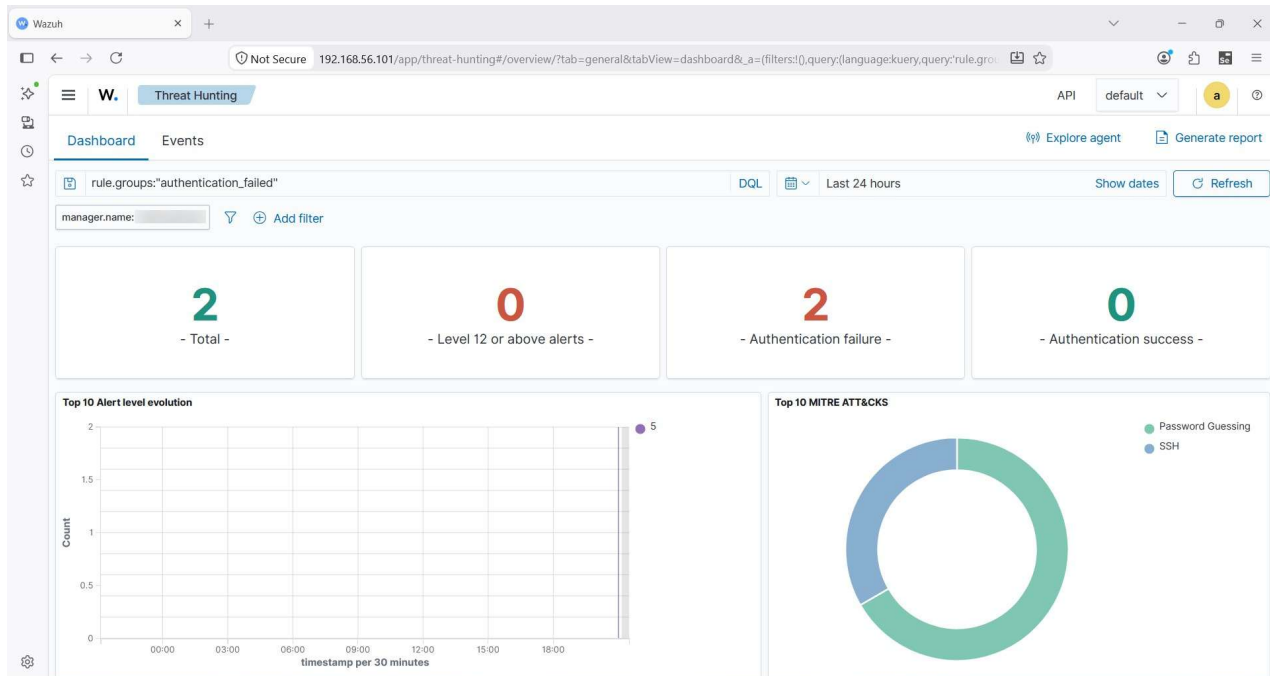


Figure 15: Wazuh detection of two authentication failures with MITRE ATT&CK Password Guessing and SSH mapping.

6.9 Windows Failed-Logon Event

A local failed-logon event was generated on Windows and verified through the Security event log. Event ID 4625 is the standard Windows indicator that an account failed to log on.

```
runas /user:.\SOCFakeUser cmd
Get-WinEvent -FilterHashtable @{LogName='Security';Id=4625} -MaxEvents 5 | Format-Table
TimeCreated,Id,Message -Wrap
```

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> runas /user:.\SOCFakeUser cmd
Enter the password for .\SOCFakeUser:
RUNAS ERROR: Unable to acquire user password
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security';Id=4625} -MaxEvents 5 | Format-Table TimeCreated,Id,Message -Wrap

TimeCreated      Id Message
-----
3/21/2026 12:17:35 AM 4625 An account failed to log on.

Subject:
  Security ID:      S-1-5-18
  Account Name:     DESKTOP-58MPPPO$
  Account Domain:   WORKGROUP
  Logon ID:         0x3E7

Logon Type:      2

Account For Which Logon Failed:
  Security ID:      S-1-0-0
  Account Name:     -
```

Figure 16: Windows Security Event ID 4625 confirming a failed logon event.

6.10 Incident Response, Mitigation, and Re-test

To demonstrate a basic SOC response action, the Kali source address was blocked from reaching SSH on the Ubuntu system using UFW. The rule was inserted before the broader allow rules so that the source-specific denial took precedence.

```
sudo ufw insert 1 deny from 192.168.56.102 to any port 22 proto tcp
sudo ufw status numbered
```

```
sudo ufw insert 1 deny from 192.168.56.102 to any port 22
proto tcp
sudo ufw status numbered
Rule inserted
Status: active
```

	To	Action	From
	--	-----	----
[1]	22/tcp	DENY IN	192.168.56.102
[2]	53	ALLOW IN	Anywhere
[3]	80/tcp	ALLOW IN	Anywhere
[4]	443/tcp	ALLOW IN	Anywhere
[5]	22/tcp	ALLOW IN	Anywhere
[6]	443/tcp	ALLOW IN	192.168.56.0/24
[7]	1514/tcp	ALLOW IN	192.168.56.0/24
[8]	1515/tcp	ALLOW IN	192.168.56.0/24
[9]	22/tcp	ALLOW IN	192.168.56.0/24
[10]	53 (v6)	ALLOW IN	Anywhere (v6)
[11]	80/tcp (v6)	ALLOW IN	Anywhere (v6)
[12]	443/tcp (v6)	ALLOW IN	Anywhere (v6)
[13]	22/tcp (v6)	ALLOW IN	Anywhere (v6)

Figure 17: UFW deny rule blocking Kali IP 192.168.56.102 from TCP/22.

The SSH connection was repeated from Kali after mitigation. The result was a connection timeout, confirming that the defensive rule was effective.

```
ssh socdemo@192.168.56.101
```

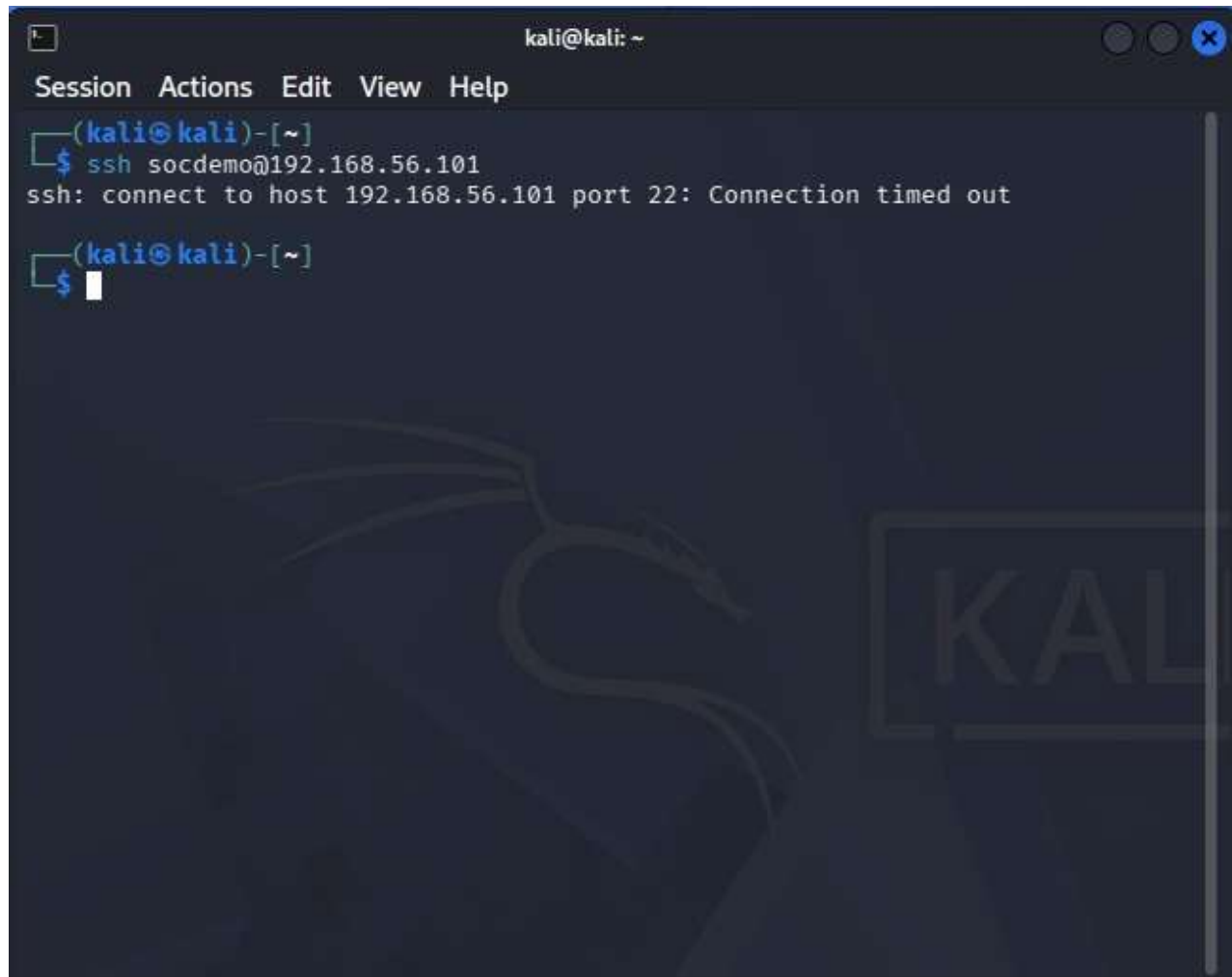


Figure 18: Post-mitigation SSH re-test from Kali resulting in a connection timeout.

7. Results and Discussion

The implemented laboratory successfully demonstrated the main operational stages expected from a small SOC: collection, detection, visualization, investigation, response, and validation. The results are summarized below.

Test / Capability	Evidence	Detection Result	SOC Value	Outcome
Windows agent enrollment	Fig. 5	Endpoint active	Centralized endpoint visibility	Successful
Sysmon telemetry	Figs. 6-7	23 Sysmon-related events visible	Rich host telemetry and MITRE mapping	Successful
File modification	Fig. 8	FIM level-7 event, Rule 550	Integrity monitoring	Successful
Kali reconnaissance	Figs. 12-13	Suricata / explicit SYN-scan detection	Network threat visibility	Successful
SSH password guessing	Fig. 15	2 authentication failures	Credential-access monitoring	Successful
Windows failed logon	Fig. 16	Event ID 4625 generated	Endpoint authentication evidence	Successful
UFW response	Figs. 17-18	SSH timed out after deny rule	Mitigation and validation	Successful

7.1 Key Findings

- The all-in-one Wazuh deployment was sufficient for a small laboratory and provided one interface for endpoint, authentication, FIM, and IDS events.
- Sysmon significantly increased the amount and context of Windows telemetry visible in Wazuh.
- Wazuh FIM detected the controlled change to important.txt and recorded the modification as a level-7 integrity event.
- Suricata observed activity from the Kali VM and its EVE JSON output could be integrated into the Wazuh pipeline.
- Failed SSH authentications were detected and categorized as password guessing / SSH activity, consistent with MITRE ATT&CK credential-access behavior.
- The UFW response action changed the system state from reachable SSH to a timed-out connection, providing a clear before-and-after mitigation result.

8. Challenges and Resolutions

Challenge	Impact	Resolution
TCP/443 was initially occupied by an older Apache service.	Wazuh dashboard could not bind to its HTTPS port.	Apache was stopped/disabled so that Wazuh could use TCP/443.
The initial 25 GB Ubuntu virtual disk became insufficient during dashboard installation.	The installation stopped before the dashboard completed.	The VDI was expanded to 50 GB and the Linux partition/filesystem were extended.
Repeated failed installation attempts left partial Wazuh package state.	Package-manager errors prevented clean reinstall.	Broken packages were cleaned and a fresh all-in-one installation was performed.
The first Kali virtual disk extraction produced a GRUB boot failure.	The attack/test VM did not boot.	The official pre-built archive was re-extracted and re-added to VirtualBox.
The Windows failed-logon event was clearly visible locally but was not retained as a separate Wazuh screenshot.	One optional SOC screenshot was omitted.	The local Event ID 4625 evidence was retained; other Wazuh detections (Sysmon, FIM, SSH, Suricata) provided centralized SOC evidence.

9. Security Recommendations

- Keep Wazuh manager, indexer, dashboard, agents, Sysmon, and Suricata updated to supported versions.
- Use strict firewall rules and restrict Wazuh management ports to trusted management networks rather than broad source ranges.
- Use strong passwords, account lockout or conditional-access policies, and multi-factor authentication where supported to reduce password-guessing risk [4].
- Use centrally managed Sysmon configuration with event filtering to improve signal quality and reduce unnecessary event volume.
- Monitor sensitive directories through FIM and investigate unexpected checksum or content changes promptly.
- Maintain tested Suricata rules and review EVE alerts in the SIEM so network detections can be correlated with endpoint activity.
- Create documented incident-response playbooks for scanning, brute-force attempts, suspicious file changes, and compromised endpoints.
- For a production environment, separate Wazuh central components and use proper certificates, backups, access control, and high-availability design instead of a single laboratory node.

10. Conclusion

The project successfully implemented a Mini Security Operations Center in an isolated VirtualBox laboratory and demonstrated an end-to-end security monitoring and response workflow. Wazuh provided centralized event analysis and visualization, the Windows agent and Sysmon supplied endpoint telemetry, FIM identified file changes, and Suricata supplied network IDS events. Controlled Kali-based reconnaissance and SSH authentication tests generated observable security evidence, while a Windows Event ID 4625 demonstrated failed-logout auditing on the monitored endpoint. Most importantly, the project did not end at detection: a UFW rule was applied to block the test source, and a subsequent SSH attempt timed out, validating the defensive response. The final system therefore satisfies the practical goal of showing how a SOC can move from visibility to detection, investigation, mitigation, and verification while maintaining ethical and legal safety boundaries.

11. References

- [1] Wazuh, "Architecture - Getting started with Wazuh," Wazuh Documentation.
<https://documentation.wazuh.com/current/getting-started/architecture.html> (accessed Aug. 21, 2026).
- [2] Microsoft, "Sysmon - Sysinternals," Microsoft Learn, updated Jun. 17, 2026. <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon> (accessed Aug. 21, 2026).
- [3] Open Information Security Foundation (OISF), "EVE JSON Output," Suricata Documentation.
<https://docs.suricata.io/en/latest/output/eve/eve-json-output.html> (accessed Aug. 21, 2026).
- [4] MITRE, "Brute Force: Password Guessing (T1110.001)," MITRE ATT&CK.
<https://attack.mitre.org/techniques/T1110/001/> (accessed Aug. 21, 2026).
- [5] Wazuh, "Packages list - Installation guide," Wazuh Documentation.
<https://documentation.wazuh.com/current/installation-guide/packages-list.html> (accessed Aug. 21, 2026).
- [6] CSE 487 - Cyber Security, Law, and Ethics, "Final Assignment Guidelines: Ethical Hacking Project," Department of Computer Science and Engineering, East West University, 2026.

Appendix A - Core Command Summary

This appendix lists only the primary commands required to reproduce the completed implementation. Troubleshooting commands used during failed installation attempts are intentionally excluded.

Ubuntu - Wazuh

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
sudo systemctl is-active wazuh-manager
sudo systemctl is-active wazuh-indexer
sudo systemctl is-active wazuh-dashboard
```

Windows - Wazuh Agent

```
Test-NetConnection 192.168.56.101 -Port 1514
Test-NetConnection 192.168.56.101 -Port 1515
Test-NetConnection 192.168.56.101 -Port 443
Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi" -OutFile
"wazuh-agent-4.14.7-1.msi"
Start-Service WazuhSvc
```

Windows - Sysmon and FIM

```
.\Sysmon64.exe -accepteula -i  
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5  
New-Item -ItemType Directory -Path C:\SOC-Demo -Force  
Add-Content C:\SOC-Demo\important.txt "Controlled SOC modification"
```

Ubuntu - Suricata

```
sudo apt install -y suricata  
sudo suricata-update  
sudo suricata -T -c /etc/suricata/suricata.yaml  
sudo suricata -D -i enp0s8 -c /etc/suricata/suricata.yaml
```

Kali - Reconnaissance and SSH Test

```
ip -br a  
ping -c 3 192.168.56.101  
sudo nmap -sS -Pn -T4 --top-ports 1000 192.168.56.101  
ssh socdemo@192.168.56.101
```

Ubuntu - Mitigation

```
sudo ufw insert 1 deny from 192.168.56.102 to any port 22 proto tcp  
sudo ufw status numbered
```